

Article

RVR Blockchain Consensus: A Verifiable, Weighted-Random, Byzantine-Tolerant Framework for Smart Grid Energy Trading

Huijian Wang ¹, Xiao Liu ^{1,*} and Jining Chen ²

¹ Guangxi Key Laboratory of Multimedia Communications and Network Technology, Guangxi Colleges and Universities Key Laboratory of Multimedia Communications and Information Processing, School of Computer, Electronics and Information, Guangxi University, Nanning 530004, China; 2213393044@st.gxu.edu.cn

² Guangxi Key Laboratory of Digital Infrastructure, Guangxi Zhuang Autonomous Region Information Center, Nanning 530000, China; 2112301001@st.gxu.edu.cn

* Correspondence: xiaoliu20212021@163.com; Tel.: +86-18890038296

Abstract: Blockchain technology empowers decentralized transactions in smart grids, but existing consensus algorithms face efficiency and security bottlenecks under Byzantine attacks. This article proposes the RVR consensus algorithm, which innovatively integrates dynamic reputation evaluation, verifiable random function (VRF), and a weight-driven probability election mechanism to achieve (1) behavior-aware dynamic adjustment of reputation weights and (2) manipulation-resistant random leader election via VRF. Experimental verification shows that under a silence attack, the maximum latency is reduced by 37.88% compared to HotStuff, and under a forking attack, the maximum throughput is increased by 50.66%, providing an efficient and secure new paradigm for distributed energy trading.

Keywords: blockchain; smart grid; byzantine fault tolerance; verifiable random function (VRF); reputation mechanism; RVR consensus



Academic Editors: Caterina Tricase, Radu State, Nino Adamashvili and Roberto Tonelli

Received: 25 March 2025

Revised: 30 May 2025

Accepted: 1 June 2025

Published: 13 June 2025

Citation: Wang, H.; Liu, X.; Chen, J. RVR Blockchain Consensus: A Verifiable, Weighted-Random, Byzantine-Tolerant Framework for Smart Grid Energy Trading. *Computers* **2025**, *14*, 232. <https://doi.org/10.3390/computers14060232>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

As a fundamental component of the emerging energy paradigm, the smart grid amalgamates communication technology, information technology, and power infrastructure [1–3], enabling intelligent coordination across energy production, transmission, and consumption domains. Its decentralized energy trading model effectively orchestrates distributed energy resources [4], such as electric vehicles, thereby enhancing the utilization of renewable energy sources and minimizing operational costs [5]. Nonetheless, open transaction architectures are susceptible to security vulnerabilities, including data tampering and man-in-the-middle attacks, underscoring the pressing necessity for an efficient and resilient trust management mechanism. The decentralized characteristics and tamper-resistant properties of blockchain technology [6] present promising solutions to these security challenges [7].

This research introduces the Reputation-VRF-Roulette (RVR) consensus framework, which is designed to achieve three primary objectives:

- (1) A dynamic reputation model to evaluate behavior in real time;
- (2) A VRF-driven leader election mechanism that is resistant to Byzantine manipulation;
- (3) A universal protocol to support multi-energy transactions.

The RVR framework facilitates collaborative optimization of security and efficiency, addressing the bottlenecks associated with traditional chain consensus mechanisms.

This research addresses a fundamental challenge in the design of consensus mechanisms that effectively balance efficiency and security to facilitate distributed energy trading

within complex smart grid environments. It presents an innovative blockchain consensus paradigm that integrates both security and scalability for distributed energy transactions in smart grids. By employing dynamic reputation weights in conjunction with verifiable random functions (VRFs) [8], this approach mitigates the traditional performance limitations associated with chain consensus in the presence of Byzantine attacks, thereby providing methodological advancements to tackle the uncertainties related to node behavior in open systems. In comparison to existing frameworks such as PBFT [9] and PoW [10], the RVR framework achieves a collaborative optimization of malicious node suppression and random election for the first time, establishing a theoretical basis for high-concurrency energy trading scenarios. On a practical application level, the universal design of RVR accommodates coupled transactions involving various energy media, including photovoltaics, energy storage, and electric vehicles, thus broadening the applicability of blockchain technology within smart grids. Additionally, this framework offers reusable consensus design references for other distributed systems that require high-security measures, such as intelligent transportation systems and the industrial Internet of Things.

2. Literature Review

The equilibrium between security and communication efficiency remains a pivotal challenge in the domain of blockchain consensus research. The current literature can be classified into three primary categories: traditional Byzantine fault tolerance (BFT) algorithms, enhancements to chain-based consensus mechanisms, and optimizations tailored to specific applications. This section provides a systematic examination of their contributions and limitations, particularly in the context of energy trading within smart grids.

2.1. Traditional BFT Algorithms

The Bitcoin system designed by Satoshi Nakamoto uses the Proof of Work (PoW) consensus algorithm proposed by Jakobsson and Juels [10], which consumes a considerable amount of energy. The practical Byzantine fault tolerant (PBFT) algorithm [9] is a cornerstone of Byzantine fault tolerant systems, facilitating consensus among distributed nodes. Nonetheless, its communication complexity of $O(n^2)$ poses a considerable obstacle for deployment in large-scale networks. In light of this challenge, various optimization strategies have been proposed in subsequent research. For example, the Zyzzyva algorithm, developed by Kotla et al. [11], markedly improves consensus efficiency under normal operating conditions by distinguishing between fault-free and faulty scenarios. Building upon this foundation, the HoneyBadgerBFT algorithm, introduced by Miller et al. [12], incorporates a timing-based mechanism for handling Byzantine errors, effectively mitigating the timeliness issues associated with network latency. Furthermore, Zhang et al. [13] utilize sharding technology to divide the network into multiple committees for parallel transaction processing, thereby significantly enhancing system scalability. The Thunderella algorithm, proposed by Pass and Shi [14], innovatively employs state machine replication technology, substantially increasing transaction processing speed by reducing the repetitive identification tasks performed by honest nodes within the blockchain.

2.2. Innovations in Chain-Based Consensus

In response to scalability challenges, researchers have increasingly focused on chain-based designs. The SBFT algorithm [15] has successfully reduced communication overhead to $O(N)$ while streamlining voting processes. HotStuff [16] has optimized leader rotation through a three-stage voting mechanism, augmented by threshold signature technology [17], achieving linear complexity during view switching. FastHotStuff [18] has further introduced aggregated quorum certificates (aggQCs) to minimize latency. However,

these algorithms exhibit vulnerabilities in leader selection when subjected to Byzantine attacks, such as forking and silence attacks [19], which constrain their applicability in open peer-to-peer (P2P) energy trading systems.

2.3. Challenges Specific to Smart Grids

Although blockchain-based solutions for energy trading—such as token mechanisms [20] and decentralized charging [21]—have been explored, the majority of studies tend to focus on single-device scenarios or lack comprehensive frameworks. For instance, while Kumar et al. [22] proposed a hybrid EV trading model, their approach lacked dynamic reputation management mechanisms. Additionally, current consensus protocols encounter difficulties in balancing efficiency and security within multi-entity smart grids, particularly when malicious nodes disrupt leader elections.

2.4. Research Gaps

The existing literature reveals two notable gaps:

1. Insufficient Byzantine-Resistant Leader Selection: The leader rotation mechanisms inherent in traditional chain consensus are vulnerable to targeted attacks.
2. Scenario-Specific Limitations: A predominant focus on isolated energy types or devices restricts the applicability of findings to heterogeneous smart grid environments.

Our proposed RVR consensus framework seeks to address these gaps by integrating dynamic reputation evaluation with verifiable randomness, thereby ensuring both security and scalability in multi-energy trading contexts.

3. Methodology

3.1. Research Design

The primary aim of this research is to establish a blockchain consensus framework [23] that adeptly reconciles efficiency and security to promote decentralized energy trading within smart grids, particularly in the face of Byzantine attacks. The research methodology is organized into three distinct phases. First, a dynamic reputation model is developed to evaluate the trustworthiness of nodes in real-time, employing multi-dimensional behavioral metrics such as node activity, voting contribution, and compliance with behavioral norms. Next, a verifiable random function (VRF) is integrated to generate tamper-resistant global random numbers, which serve as the foundation for a reputation-weighted roulette wheel election mechanism [24]. Finally, simulation experiments are conducted to assess the performance advantages of the proposed framework in scenarios characterized by forking and silence attacks.

3.2. Dynamic Reputation Model

The dynamic reputation model is constructed to continuously recalibrate node trust values by formalizing multi-dimensional behavioral data. Node activity is quantified via nonlinear aggregation of the regional user base and transaction throughput, thereby reflecting real-time load conditions. Voting contribution is evaluated based on the number of valid votes cast, which quantifies participation in the consensus process. A behavioral reward-penalty mechanism is implemented to dynamically adjust reputation weight, applying exponential penalties for negative behaviors (e.g., timeout incidents) and linear rewards for positive actions (e.g., timely operations), thus effectively diminishing the impact of malicious nodes. The comprehensive reputation score integrates historical reputation data with real-time behavioral evaluations, ensuring that nodes with elevated reputations are prioritized during elections.

3.3. VRF-Driven Leader Election

The VRF-driven leader election mechanism is based on the generation of tamper-proof random numbers using the verifiable random function, which are combined with dynamic reputation weights to achieve a balance between fairness and security. The process is outlined as follows: (1) a third-party authority distributes the consensus private key; (2) nodes generate random numbers along with their corresponding verification proofs based on the current view value; (3) after independent validation of the random numbers, a roulette wheel algorithm is utilized to map reputation weights to probability intervals (for example, a node with a 30% reputation weight occupies the interval $[0, 0.3)$). This design ensures that election outcomes remain unpredictable while simultaneously prioritizing nodes with high reputations, thereby effectively countering Byzantine manipulation.

4. Security Model of Transaction System

4.1. System Model

The blockchain-based smart grid energy trading framework presented in this research consists of four interconnected layers: the entity layer, which enables the authentication of identities for both producers and consumers through unique identifiers (IDs) and public-private key pairs for energy transactions (Figure 1); the energy layer, which manages the injection and extraction of energy via distributed energy warehouses [25], dynamically authorized by service nodes to ensure the legitimacy of energy transmission; the service layer, which oversees transaction processing through various mechanisms, including information routing [26], Byzantine fault tolerance protocols [8], dual signature verification [27], payment amount matching [28], and block pool storage [29]; and the blockchain layer, which is collaboratively maintained by nodes to sustain an immutable distributed ledger. Within this framework, users at the entity layer (for example, Entity A and Entity B) connect to the distributed warehouses of the energy layer through designated energy interfaces. When Entity A initiates a transaction request, which includes its ID and a private key signature, the server disseminates this request throughout the network. In response, Entity B provides a signed confirmation, which the server validates to authenticate the identities involved [30]. Following this validation, the server instructs the energy layer to facilitate the transfer of energy from Entity B's warehouse to Entity A's through established energy links [25], with all transaction data being permanently recorded on the blockchain to ensure integrity and traceability.

4.1.1. Entity Layer

The composition unit includes transaction entities such as producers (e.g., power plants) and consumers (e.g., households and factories), which is defined as follows: $EL = \{E_1, E_2, \dots, E_n\}$. The metadata structure of each Entity E_i is expressed as follows:

$$E_i = \{ID_i, PK_i, SK_i, balance, Energy\ Interface\}$$
 (1)

where ID_i serves as a unique identifier that distinguishes various entities, such as power plants and users. PK_i refers to the public key utilized for encrypting data and verifying signatures, while SK_i denotes the private key employed for generating signatures and decrypting operations [30]. The term indicates the energy or currency balance of an account. The energy interface functions as an energy exchange interface that facilitates energy storage, extraction, and trading operations. The primary role of the entity layer is to interact with the energy warehouse via energy interfaces and to execute energy storage and transaction operations.

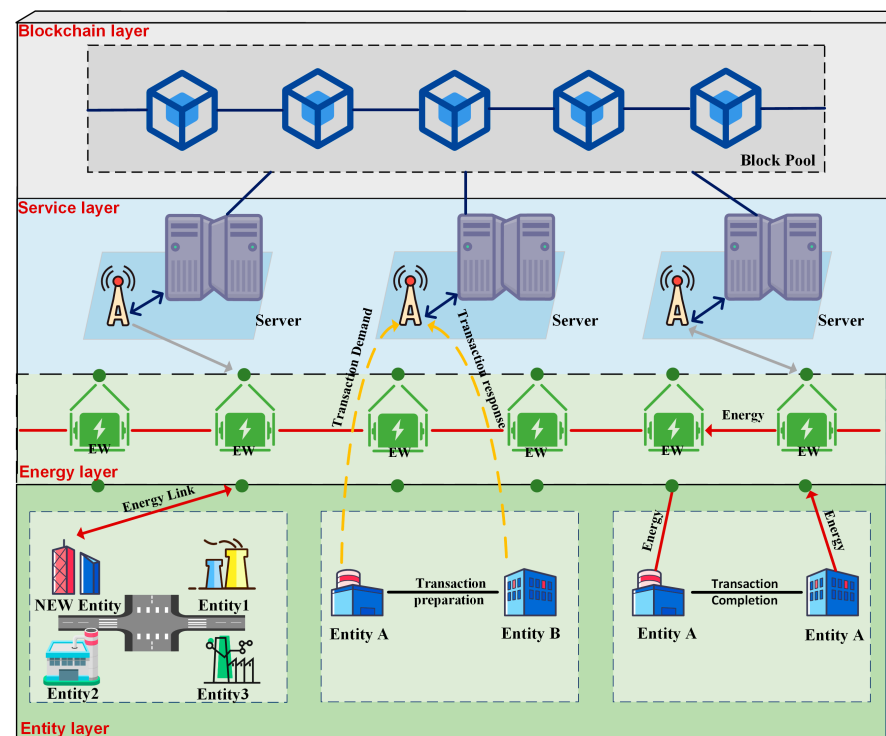


Figure 1. The blockchain-based smart grid transaction framework.

4.1.2. Energy Layer

An energy warehouse (EW) [25] is a distributed storage node that connects multiple entities to facilitate energy flow. It involves producer injection ($EW \leftarrow EQ_{produce}$) and consumer extraction ($EW \leftarrow EQ_{consume}$). The service node dynamically authorizes the interface permissions, and the verification equation is expressed as follows:

$$\text{Access}(E_i) = \text{Verify}(SK_i, PK_{SN}) \quad (2)$$

where PK_{SN} represents the public key of the service node, it is utilized to verify the legitimacy of the signature. This function confirms the authenticity of the entity's identity by matching the correspondence between the private and public keys, thereby authorizing access operations.

4.1.3. Service Layer

The service layer mediates interactions between entities and the blockchain, ensuring transaction integrity and consensus execution. As depicted in Figure 2, it comprises four synchronized modules:

The information routing module (IM) forwards transaction requests from the Entity Layer to the consensus network.

The consensus control module (CM) implements the byzantine fault tolerance (BFT) protocol to ensure the integrity of the blockchain's state.

The transaction verification module (VM) confirms the validity of the dual signature, ensuring that the energy quantity (EQ) corresponds to the payment amount (k).

The block pool storage (BP) verifies the transaction data and awaits block packaging.

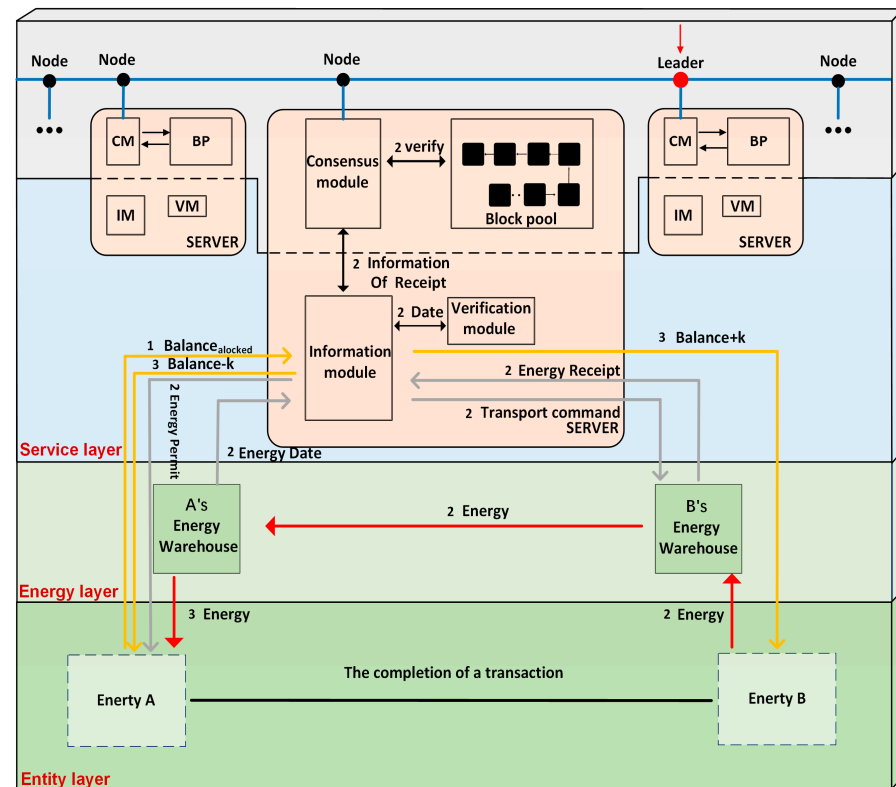


Figure 2. Transaction flowchart.

4.1.4. Blockchain Layer

The blockchain layer functions as a decentralized and tamper-resistant distributed ledger [23], which is upheld by a network of nodes. It systematically documents transaction data within cryptographically interconnected blocks, employs digital signatures to verify authenticity, and incorporates an optimized architecture to ensure data integrity and secure synchronization in the context of energy trading.

4.2. Transaction Process

Transactions implement a two-phase atomic commitment protocol where Phase 1 (negotiation) establishes binding agreements via digitally signed offers/acceptances (Figure 1), and Phase 2 (execution) enforces settlement through pre-frozen funds verification, cryptographic energy escrow, and automated blockchain clearing (Figure 2), resolving the temporal asynchronicity between energy and financial transactions while guaranteeing atomicity and non-repudiation.

4.2.1. Transaction Negotiation

Energy consumers initiate transactions by broadcasting digitally signed requests containing their ID and energy demand (Figure 1). Producers respond with signed commitments that undergo service-layer signature verification, establishing binding agreements before execution.

$$TX_Demand = \{ID_A, sign_A(SK_A), EQ_{req}\} \quad (3)$$

Here, ID_A serves as the unique identifier for the transaction initiator (such as a user or power plant), and $sign_A(SK_A)$ represents a digital signature generated using a private key, which is used to verify identity and transaction legality. EQ_{req} denotes the requested amount of energy (such as electricity consumption or generation), facilitating transaction matching and settlement.

After confirming the intention to transact, Entity B sends a transaction response that includes a signed commitment.

$$TX_Commit = \{ID_B, sign_B(SK_B)\} \quad (4)$$

The verification module confirms participant identities by cryptographically validating attached signatures against registered public keys, ensuring that only authenticated entities proceed to transaction execution.

$$Verify(PK_A, sign_A) \wedge Verify(PK_B, sign_B) \quad (5)$$

The verification function is responsible for validating the signature [30] by employing a public key. It is essential that both parties successfully complete the verification process for the transaction to be considered valid. Additionally, the protocol for executing the transaction should only commence after the successful completion of identity verification.

4.2.2. Transaction Execution

The transaction execution phase constitutes the core operational workflow for energy settlement, ensuring atomicity and non-repudiation through cryptographic verification and distributed ledger coordination. As depicted in Figure 2, this process implements a three-stage commit protocol that synchronizes energy transfer with financial settlement while mitigating double-spending risks:

(1) Pre-Freezing of Funds

Prior to energy release, the service layer securely locks the specified transaction amount, creating a temporary balance reserve. This financial safeguard prevents overspending during pending transactions while maintaining account transparency, with automated rollback mechanisms activating if confirmation timeouts occur.

$$balance_A^{locked} = balance_A - k \quad (6)$$

Here, $balance_A$ represents the original account balance of Entity A, k denotes the amount that needs to be pre-frozen during the transaction, and $Balance_A^{locked}$ refers to the remaining operable balance after the funds have been locked. This mechanism ensures fund safety before transaction completion, thereby preventing overpayments or duplicate transactions. If the timeout is not confirmed, the system will automatically roll back the transaction.

(2) Energy Mortgage

Producers physically deposit committed energy quantities into designated warehouses, triggering cryptographic receipt generation with timestamped verification data. These digitally notarized receipts then initiate automated energy transfers between warehouses while preserving transaction privacy through hashed quantity records.

$$EnergyReceipt = \{t, H(EQ_{commit}), PK_B\} \quad (7)$$

Here, t represents the timestamp that records the time of transaction generation to ensure timeliness. $H(EQ_{commit})$ denotes the hash value of the committed energy quantity (EQ_{commit}), which safeguards data privacy and verifies integrity.

Subsequently, the service node transmits a command to B's energy warehouse, facilitating the transfer of energy from B's warehouse to A's energy warehouse. A's energy warehouse then receives the energy and sends the corresponding data to the server.

The service packages the credentials and energy data into information blocks, which are then authenticated and uploaded to the blockchain. Once the blockchain submission is successful, CM sends a receipt to IM, who receives the receipt to finalize the transaction.

(3) Transaction Settlement

The settlement phase finalizes the energy exchange by atomically transferring pre-frozen funds to the producer while simultaneously releasing energy access rights to the consumer.

$$balance_B \leftarrow balance_B + k \quad (8)$$

$$balance_A^{\text{locked}} \rightarrow balance_A \quad (9)$$

This coordinated asset transfer is executed through blockchain-verified balance adjustments, where the consumer receives cryptographically signed energy withdrawal credentials, enabling immediate resource retrieval and completing the transaction lifecycle with guaranteed atomicity.

$$\text{EnergyPermit} = \{PK_A, EQ_{\text{commit}}\} \quad (10)$$

4.3. Chained Leader Consensus Models and Attack Models

The chained leader consensus achieves linear communication complexity through sequential leader rotation, overcoming traditional BFT algorithms' scalability limitations via three core mechanisms: temporary leader election, chained block confirmation (requiring three subsequent blocks for finality), and view timeout switching protocols. However, these protocol features can be legitimately exploited by Byzantine nodes through forking attacks (malicious creation of competing chains) and silence attacks (intentional block submission delays), prompting the formalization of consensus rules and attack models in this section, which directly motivate RVR's reputation-based defense mechanism as described in Section 5.

4.3.1. Chained Leader Consensus Algorithm

This section provides a brief overview of the chain leader consensus algorithm's fundamental process, which enables efficient Byzantine fault tolerance through decentralized coordination. The protocol achieves linear communication complexity via sequential leader rotation while maintaining transaction finality. Its three-phase workflow establishes a robust foundation for distributed ledger consistency [16,18].

(1) Voting Stage

Replica nodes participate in leader selection by submitting votes to the randomly elected coordinator node [16,18,19]. This distributed voting mechanism ensures decentralized consensus input while preventing single-point control failures.

(2) Block Packaging

The elected leader consolidates collected votes into a quorum certificate (QC), then packages this cryptographic proof with validated transaction requests into a new block [16,18,19]. This dual-component structure provides both consensus legitimacy and transactional payload.

(3) Block Addition

The leader appends the newly created block to the terminal position of the existing blockchain [16,18,19]. This sequential chaining maintains chronological integrity while enabling deterministic block verification through hash-linked references.

To mitigate the influence of Byzantine nodes and network asynchrony, the final submission of block B_V must satisfy the requirement of continuously adding three new blocks thereafter (Figure 3).

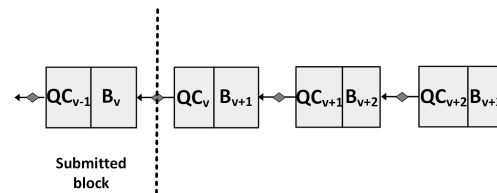


Figure 3. Successful submission of block B_V .

In addition, the system operates under the following three fundamental assumptions [16,18].

Assumption 1. Honest nodes strictly adhere to protocol specifications and are dedicated to maintaining consensus mechanisms, while malicious nodes may disrupt the consensus process.

Assumption 2. The proportion of honest nodes in the system must exceed that of malicious nodes, thereby satisfying the constraint that the total number of nodes ($n > 3f + 1$). This condition provides a foundation for consensus security.

Assumption 3. The encryption algorithm possesses theoretical security, and the digital signature mechanism is unforgeable, thereby ensuring the authenticity and trustworthiness of communication between nodes.

4.3.2. Byzantine Attack Model

Delays in verification processes may present opportunities for nefarious leader nodes to exploit vulnerabilities. While the system employs hash algorithms and public-private key authentication mechanisms to safeguard against tampering with published transaction data, attackers can exploit temporal discrepancies in verification to initiate attacks and disrupt consensus among replicas. The compliance of such attack behaviors at the protocol level often renders it difficult for external observers to detect and identify. Nonetheless, these attacks can substantially impair the performance of blockchain networks. There exist multiple strategies for attacking Byzantine systems, and this discussion will primarily concentrate on two specific methods.

Forking Attack

The Byzantine leader node may intentionally create a fork in the chain consensus algorithm to override the blocks proposed by honest nodes. The figure illustrates that the Byzantine leader node extends B_{V+3} from block B_V . Since B_{V+3} meets the voting criteria [16,18], the replica nodes will vote in favor of B_{V+3} , leading all subsequent leader nodes to expand based on B_{V+3} .

The Byzantine leader node encompasses the blocks proposed by honest nodes through a manufacturing chain fork, extending from block B_V to generate the competing block B_{V+3} . Since B_{V+3} adheres to the voting rule [16,18], replica nodes will cast their votes in favor of it, ultimately resulting in subsequent leader nodes continuing to extend based on B_{V+3} , as illustrated in Figure 4.

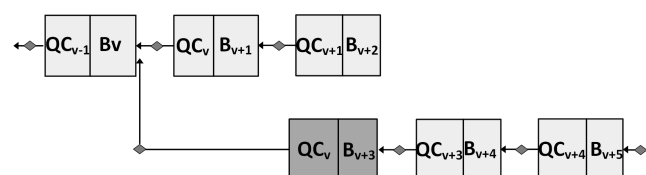


Figure 4. Forking attack.

Silence Attack

The Byzantine leader node delays block submission by issuing a negative response. When the round reaches B_{v+2} , the Byzantine leader halts block generation until the timeout mechanism for view $v+2$ is activated. Consequently, the new leader is compelled to resubmit the block after B_v , as illustrated in Figure 5.

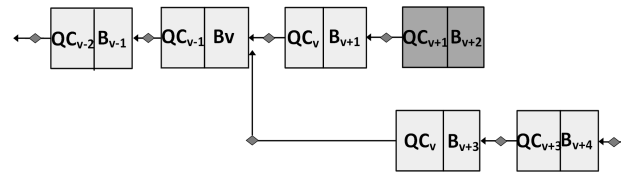


Figure 5. Silence attack.

Traditional consensus algorithms are susceptible to the duplicate selection of Byzantine nodes due to inadequate processing, which directly leads to a significant decline in the network's overall performance. For example, under a silence attack scenario (Figure 6), the performance exhibits an upward trend as the number of Byzantine nodes increases.

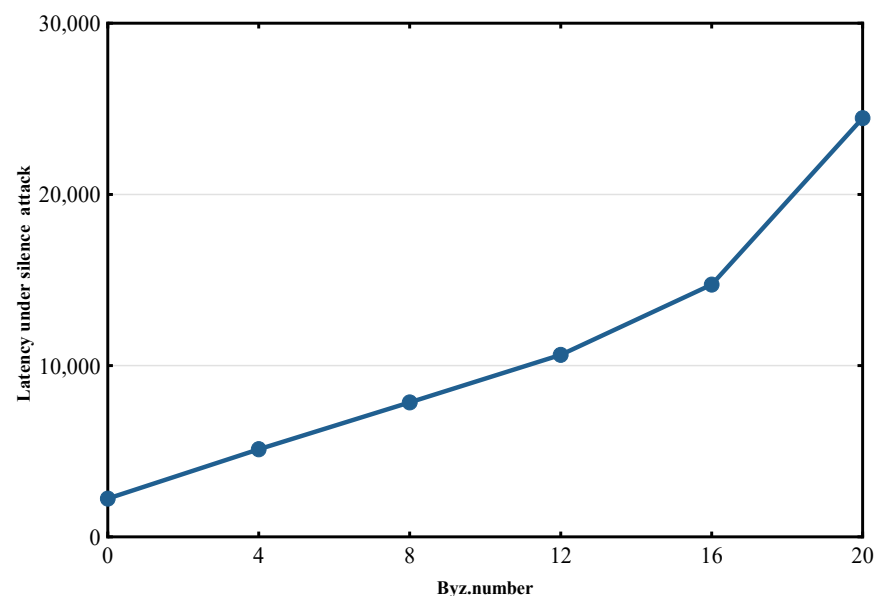


Figure 6. Network latency performance under silence attack.

To address this issue, this paper proposes the RVR consensus algorithm, which integrates a reputation evaluation system with a VRF wheel selection algorithm to effectively reduce the probability of Byzantine nodes being elected. Consequently, the algorithm achieves a significant enhancement in the performance of blockchain networks under Byzantine attacks. The specific implementation of the RVR algorithm will be discussed in the following section.

5. Chained Consensus Algorithm Based on Reputation

In response to the challenges posed by the existing leader-based consistency mechanism [9,16,18], which significantly impairs network performance during certain types of network attacks, this paper introduces a robust consistency algorithm referred to as RVR (Reputation-VRF-Roulette Consensus). This algorithm employs an evaluation model and establishes a dynamic reputation assessment system that integrates multi-dimensional node attributes and behavioral data. Furthermore, it incorporates a verifiable random function (VRF) alongside an adaptive roulette wheel selection mechanism to facilitate

probabilistic leader elections based on the weighted reputation of nodes. As illustrated in Figure 7, the algorithm's process is delineated into three fundamental stages: the construction of the reputation evaluation group, the dynamic election of leaders, and the confirmation of blockchain transactions.

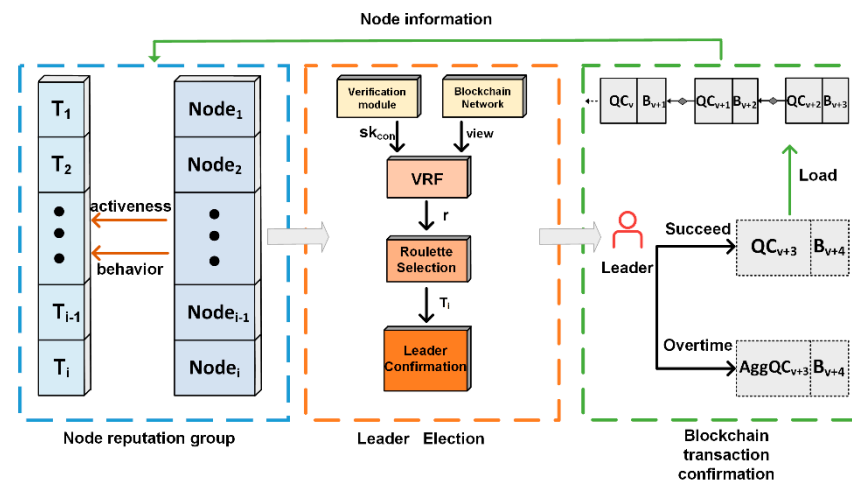


Figure 7. RVR algorithm model.

The implementation process is delineated as follows. Initially, reputation modeling is executed through the observation of multi-dimensional behavioral indicators associated with nodes. During the key management phase, the system employs a consensus private key, which is issued by a third-party trust institution, serving as the encryption seed for the verifiable random function (VRF). As the blockchain view version progresses, the VRF mechanism integrates the current block height with the consensus key to produce a distinct, auditable random parameter, denoted as r . Subsequently, a matrix calculation is conducted to determine the consensus coordination node for the current view period, utilizing an enhanced roulette wheel selector alongside the dynamic reputation weights of each node. The node that is selected assumes the responsibility of packaging the transaction set into blocks and disseminating these blocks throughout the peer-to-peer (P2P) network, thereby facilitating the update of the distributed ledger.

5.1. The Relationship Among Reputation, VRF, and Roulette

The reputation mechanism serves as the foundation of trust, dynamically calculating each node's comprehensive reputation weight T_i by monitoring nodes' activities, voting contributions, and behavioral compliance in real time (Equations (11)–(18)). Nodes with high reputations receive greater election weights due to their historical contributions and real-time performance. In contrast, malicious nodes experience a rapid decline in their reputation weight as a result of behavioral punishment mechanisms, which reduce their probability of being selected.

VRF generates a globally consistent and unpredictable random number r (Equation (19)) using the consensus private key sk_{con} and the current view value $view$, accompanied by a verifiable encryption proof π (Equation (21)). All nodes independently generate and verify the same random number, which prevents malicious nodes from manipulating the random source and ensures the transparency of the election process.

Roulette selection balances weight and randomness, combining the dynamic reputation weight T_i of nodes with the random number r generated by VRF and determining the leader through probability mapping (Equation (20)). Specifically, the election probability p_i of each node is proportional to its reputation weight, while the random number r from the VRF establishes the final selection interval. For instance, if the weight of node A accounts

for 30%, its corresponding sector on the roulette wheel is $[0, 0.3)$. Node A is selected when r falls within this interval. This design provides high-reputation nodes with a significant advantage. However, the element of randomness prevents the monopolization of fixed nodes, and the unpredictability of the VRF ensures that attackers cannot anticipate or manipulate the election results. The synergistic effect of VRF and the roulette wheel together constitutes the leader election mechanism of RVR. Specifically, VRF provides anti-manipulation random seeds to ensure the fairness of the election process. Meanwhile, the roulette wheel converts dynamic weights into probability distributions, ensuring that the election results align with node contributions while incorporating an element of randomness.

5.2. Node Reputation Mechanisms

In this section, the establishment of reputation groups is explained in detail. All nodes in the network synchronize and maintain a shared reputation ledger through a consensus protocol. To better describe the reputation mechanism, the system adopts a multi-dimensional behavior perception framework that quantifies node participation into three core observation indicators: node activity, voting activity, and behavior weight. Multi-dimensional behavior indicators (node activity, voting activity, and behavior weight) are defined using Equations (11)–(18), and a reward and punishment mechanism is introduced (Equations (15) and (16)) to dynamically adjust reputation weights. The reputation weight directly influences the probability distribution of roulette wheel selection (Equation (20)), significantly increasing the probability of high-reputation nodes being selected as leaders. This design exponentially reduces the election probability of malicious nodes (as indicated by the historical reputation decay term in Equation (18)), effectively suppressing the repeated election of Byzantine nodes.

5.2.1. Node Activity

The node activity index characterizes the real-time load status of server nodes, and its value is strongly positively correlated with the frequency of transaction processing within the regional network topology. Specifically, the growth of the user base within geographic distribution units directly leads to an increase in the volume of transaction requests. This surge results in a dynamic increase in the load of node transaction verification queues, ultimately reflected in a nonlinear rise in the activity index.

$$A(i) = \min\left(\gamma(\log(K+1))^a(\log(T+1))^b, A_{max}\right) \quad (11)$$

Here, K represents the user base, T denotes the transaction throughput, and γ is the normalization coefficient. The typical parameter configurations are $a = 0.5$ and $b = 0.3$. A_{max} refers to the activity threshold constraint term, while the suppression index indicates the nonlinear dilation. The activity level of nodes is positively correlated with both the number of users and the transaction throughput in the region. This correlation is utilized to characterize the real-time load status of the nodes.

5.2.2. Voting Activeness

The voting activity index quantifies the strength of contributions from nodes participating in consensus by calculating the number of valid votes cast during the statistical period and applying appropriate weights to them.

$$V(i) = m - e^{-\alpha v} \quad (12)$$

Here, v represents the number of times a node votes to pack into QC, α is the adjustment coefficient, and m denotes the upper limit of activity. Voting activity quantifies the contri-

bution of nodes to consensus participation by weighting the number of votes cast. During the initial stage, the number of votes has a significant impact on the reputation weights, which gradually approaches a stable value.

From this, we obtain the fundamental reputation of the node, i.e.,

$$B(i) = \alpha A(i) + (1 - \alpha)V(i) \quad (13)$$

where α represents the weight coefficient, and the basic reputation is calculated by weighting the node's activity (Equation (11)) and voting activity (Equation (12)), thereby reflecting the node's overall historical performance.

5.2.3. Behavior Weight

Behavioral weight signifies that the effectiveness of node behavior, in terms of reward and punishment, is achieved through a role grading mechanism designed to create a differentiated configuration. This is specifically reflected in the higher operational authority weights assigned to leader nodes.

$$\theta = \frac{1 + (k - 1)\zeta}{k} \quad (14)$$

Here, the value of ζ can be either 0 or 1, with 1 representing the lead node. The variable k denotes the reciprocal of the weight of a standard node. Leadership nodes are assigned higher weights through a role-grading mechanism, which differentiates the operational authority weights of configuration nodes.

As mentioned earlier, forking and silence attacks are challenging to detect directly. However, we can obtain the behavior penalty $P(n)$ and the behavior reward $R(t)$ based on the behavior weight:

$$R(t) = r_0 \cdot \theta \cdot \frac{1}{1 + \beta \cdot \log(1 + n)} \cdot (1 - t) \quad (15)$$

where r_0 ($1 \leq r_0 \leq 10$) is the basic reward value. It is essential to balance the intensity of incentives with the stability of the system. The parameter β ($0.1 \leq \beta \leq 0.3$) serves as the attenuation coefficient, which controls the decay rate of rewards in relation to the number of nodes, thereby preventing excessive dilution of rewards. The variable t denotes time, where $t = 1$ indicates a timeout and $t = 0$ signifies punctuality. Additionally, n represents the total number of timeout occurrences.

$$P(n) = p_0 \cdot \theta \cdot (1 + \alpha \cdot n^2) \quad (16)$$

Here, p_0 is the basic penalty value, and it is recommended that p_0 be set between $0.2r_0$ and $0.3r_0$. It is important to ensure that the penalty is sufficiently strong to deter Byzantine attacks. The variable α ($0.1 \leq \alpha \leq 0.5$) represents the punishment growth coefficient. This coefficient regulates the growth rate of the penalty based on the number of timeouts.

If a node is penalized for exhibiting abnormal behavior, such as experiencing a timeout, the penalty value increases exponentially with the number of timeouts incurred. Conversely, if a node receives a reward for completing an operation within the designated time frame, the reward value increases linearly with time stability. Specifically, we can derive an equation for the behavioral reputation, denoted as $C(i)$.

$$C(i) = R(t) - P(n) \quad (17)$$

Comprehensive systems of punishment and reward for behavioral reputation dynamically reflect the real-time performance of nodes.

Therefore, we synthesize the basic reputation and behavioral reputation to obtain the comprehensive reputation of the node:

$$T_n^i = T_{n-1}^i + k_1 B(i) + k_2 C(i) \quad (18)$$

where k_1 ($0.5 \leq k_1 \leq 0.7$) is the historical reputation weight, which emphasizes long-term stability and aims to minimize credit fluctuations, and k_2 ($0.3 \leq k_2 \leq 0.5$) is the weight assigned to behavioral rewards and punishments. To ensure that real-time behavior has an immediate impact on reputation, the condition $k_1 + k_2 = 1$ must be satisfied. The comprehensive reputation is calculated by weighting the basic reputation (Equation (13)) and the behavioral reputation (Equation (17)) and dynamically updating the global reputation weights of the node.

5.3. Leader Election Mechanism

Compared to the security vulnerabilities associated with the repeated election of malicious nodes in existing chained Byzantine fault tolerance (BFT) algorithms [16,18], the RVR scheme introduces a verifiable probabilistic roulette wheel election mechanism. This mechanism dynamically adjusts the real-time weights of candidate nodes through a reputation evaluation system. It leverages the dynamic perception capabilities of the chain topology structure to develop a leader election model that satisfies the consensus verification requirements of BFT. A key innovation of this model is the quantification of the historical behavioral reputation of nodes as dynamic parameters within the election probability distribution function. This approach ensures that the probability of malicious nodes being elected decreases exponentially in response to their abnormal behavior.

5.3.1. Generating VRF Random Numbers

Each node utilizes the consensus private key sk_{con} and consensus input data (consensus view) to generate random numbers and proofs:

$$(r_{global}, \pi_{global}) = \text{VRF}_{sk_{con}}(view) \quad (19)$$

A trusted third-party institution releases the consensus private key, which facilitates the generation of random numbers through three key parameters: the current view value ($view$), the binary random number generated by the node, and its corresponding verifiable random function proof. A globally consistent random number, denoted as r_{global} , is generated using the consensus private key and the current view value (Equations (19)–(21)). This design ensures that random seeds cannot be predicted or tampered with by malicious nodes, and all nodes to verify the legitimacy of the random numbers are based on the same input, thereby preventing manipulation of election results. The random numbers generated by the verifiable random function (VRF) serve as inputs for roulette wheel selection, combined with dynamic weights (Equation (20)), to ensure the fairness of the election (randomness) while giving higher priority to nodes with a strong reputation (weight-driven selection).

5.3.2. Leader Confirmation

The standard random number r is utilized as input, combined with the node's reputation weight T_i ($i = 1, 2, \dots, n$) through a roulette algorithm to elect the leader, namely:

$$\text{Leader}_i = \min \left\{ i \mid \sum_{j=1}^i T_j \geq r \times \sum_{j=1}^n T_j \right\} \quad (20)$$

where $Leader_i$ represents the selected leader node, and all nodes choose leaders based on the same common random number, ensuring consistent results.

5.3.3. Verification of Leader Election

Any other node can verify the leader's election. Since the election result is deterministically derived from the verifiable random number r , authenticating the leader election is essentially equivalent to verifying the correctness and provenance of r . The process is expressed as follows:

$$ture/false \leftarrow \text{Verify}(pk_{\text{shared}}, \text{view}, r_{\text{global}}, \pi_{\text{global}}) \quad (21)$$

5.4. RVR Algorithm Flow

In this section, we will present a comprehensive overview of the RVR algorithm process, detailing its node reputation mechanism and leader selection algorithm.

This algorithm employs a pipeline architecture, which transcends the execution paradigm of the traditional four-stage consensus model. As illustrated in Algorithm 1, each node in the network assumes a distinct role: consensus nodes generate and broadcast votes, while leader nodes concurrently execute block construction operations.

Algorithm 1: RVR Algorithm Process

```

1  As LeaderNode
2    Wait msgs from replicas
3    if n-f  $\eta$  msgs are received
4      AggQC  $\leftarrow$  CreatAggQC( $\eta$ set)
5      B  $\leftarrow$  CreateMsg(Prepare,view,aggQC,cmd,id)
6    end
7    if n-f v msgs are received
8      QC  $\leftarrow$  CreateQC(V)
9      B  $\leftarrow$  CreateMsg(Prepare,view,QC,cmd,id)
10   end
11   Broadcast B
12   Update Tarry
13   nextleader  $\leftarrow$  LeaderElection(view,Tarry)
14   SendVote(nextleader,V,view+1)
15 end
16 As ConsensNode
17   Wait for B from LeaderNode
18 if B is accepted successfully
19   Update Tarry
20   nextleader  $\leftarrow$  LeaderElection(view,Tarry)
21   Send Vote(nextleader,V,view+1)
22 end
23 if a timeout occurs at any stage
24   Update Tarry
25   nextleader  $\leftarrow$  LeaderElection(view,Tarry)
26   Send  $\eta$ (nextleader,V,view+1)
27 end
28 end

```

In the voting collection stage (Processes 3–5), the system generates AggQCs with Byzantine fault tolerance characteristics using η -type voting (refer to FastHotStuff). Upon entering the block construction stage (Processes 8–9), the algorithm constructs a quorum certificate (QC) and encapsulates the validated transaction set within the block.

Specifically, in Processes 11–14, the selected nodes broadcast new blocks to the entire network through point-to-point communication, subsequently initiating a leader replacement protocol based on dynamic reputation. The specific implementation details will be discussed in the next section.

In standard operational scenarios (Processes 18–21), consensus nodes evaluate candidate blocks and generate standard votes. When a network anomaly or view timeout event is detected (Processes 23–26), the node activates the fault-tolerant protocol and transmits a η -type data packet containing timestamp encryption to maintain system activity. This dual-mode response mechanism significantly enhances the algorithm's robustness in complex network environments while ensuring safety.

5.5. Theoretical Analysis

This section formally establishes RVR's security guarantees using mathematical proofs. The safety proof (Theorem 1) demonstrates irreversible transaction consistency by leveraging quorum certificate intersections and honest node behavior constraints. The liveness proof (Theorem 2) guarantees finite-time transaction finality by analyzing reputation-driven leader election probabilities under Byzantine failures, where malicious leader selection likelihood decreases exponentially through behavioral penalties.

5.5.1. Safety Guarantees

Theorem 1 (Consistency) [16,18]. *All honest nodes commit to the same sequence of blocks.*

Proof. Assume there are two honest nodes, N_1 and N_2 , that commit blocks B and B' at height h . Let $QC(B)$ and $QC(B')$ represent their respective quorum certificates. Based on the principle of quorum intersection, there must exist an honest node, N_3 , that signed both $QC(B)$ and $QC(B')$. This situation contradicts the protocol rule that states honest nodes can only vote for one block per height. Therefore, safety is ensured. $\square$

5.5.2. Liveness Guarantees

Theorem 2 (Progress) [16,18]. *Any valid transaction is committed within a finite time after the Global Stabilization Time (GST).*

Proof. Let T represent the maximum epoch duration (Δ + processing time). If a leader is Byzantine, a view switching will occur within T . Due to reputation decay mechanisms, the probability of Byzantine leader election decreases exponentially. Honest leaders will ultimately propose blocks, thereby ensuring liveness. $\square$

5.6. Comparative Analysis

This section provides a systematic comparison of the reputation-based voting and reputation (RVR) algorithm with established consensus algorithms, focusing on communication complexity, energy consumption, leader election mechanisms, latency, and support for chain consensus, as detailed in Table 1. The aim is to elucidate the design advantages of RVR.

Table 1. Comparison of consensus algorithms.

Algorithm	Complexity	Energy Consumption	Leader Election Mechanism	Latency	Is Chained Consensus?
RVR	$O(n)$	Low	Dynamic reputation weight random verifiable selection	Low	Yes
HotStuff [16]	$O(n)$	Low	Equal probability election	Low	Yes
FastHotStuff [18]	$O(n)$	Low	Equal probability election	Low	Yes
PBFT [9]	$O(n^2)$	Low	Fixed sequence rotation	High (large-scale)	No
SBFT [15]	$O(n)$	Low	The one with the highest reputation	Low	Yes
Pow [10]	/	Very High	Computational power competition	Very low	Yes

Communication Complexity: RVR exhibits a linear communication complexity of $O(N)$, which is markedly more efficient than the quadratic complexity of $O(N^2)$ associated with practical Byzantine fault tolerance (PBFT). A notable innovation of RVR is its integration of dynamic reputation weights with a verifiable random function (VRF)-based roulette wheel selection mechanism. This design allows for the completion of leader election in a single round of broadcasting, thereby circumventing the multi-round interactions that are characteristic of traditional Byzantine fault tolerant (BFT) algorithms. Conversely, PBFT encounters challenges in supporting large-scale network deployments due to the frequent voting and confirmation communications required among nodes.

Energy Consumption: The reduced energy consumption of RVR can be attributed to several design features:

Linear Communication Overhead: By minimizing the number of network transmissions, RVR directly reduces energy consumption associated with communication.

Efficient Election Mechanism: The generation of random numbers using VRF necessitates only lightweight encryption calculations. Furthermore, the roulette wheel selection process is efficiently conducted based on pre-computed reputation weights, thus avoiding the substantial computational demands typical of proof of work (PoW) algorithms.

Dynamic Reputation Optimization: This feature enables the real-time isolation of malicious nodes, thereby diminishing invalid consensus participation and conserving computational resources.

Leader Election Mechanism: RVR adeptly balances attack resistance and fairness through the synergistic use of dynamic reputation weights and VRF. The probability of a node being elected is positively correlated with its historical contributions, facilitating the swift removal of malicious nodes through reputation decay. In contrast, the equal probability election mechanisms utilized by HotStuff and FastHotStuff are susceptible to interference from Byzantine nodes, while PBFT's fixed rotation strategy lacks the requisite flexibility. Additionally, SBFT's approach of selecting the node with the highest reputation as the leader can lead to a single point of failure.

Latency: The low latency of RVR is primarily due to its dynamic reputation mechanism, which minimizes the frequency of view switching. The deterministic random number generation provided by VRF eliminates the overhead associated with conflict retries that are common in traditional random election processes. In contrast, PBFT significantly increases latency in large-scale networks due to its heightened communication complexity.

Chain Consensus Support: Both RVR and HotStuff support chain structures to ensure transaction finality. However, PBFT's design limitations prevent it from adapting to a chain architecture, thereby restricting its applicability in smart grid scenarios.

5.7. Experimental Setup

This research employed the Golang programming language to create a distributed simulation trading environment. Experimental results represent mean values from five independent trials. The experimental setup is characterized by the deployment of each replica node on separate virtual machine instances, each allocated with independent memory space. The inter-node communication delay is maintained at or below 1 millisecond, and client programs are executed on distinct virtual machines. During the testing phase, clients randomly select replica nodes to initiate requests, while the designated leader node is tasked with replicating and disseminating these requests to the remaining nodes. The experiment specifically simulated Byzantine fault-tolerant scenarios by incorporating faulty nodes that exhibit Byzantine behavior. To assess the performance characteristics of the RVR algorithm, the HotStuff and FastHotStuff algorithms were concurrently implemented within the same experimental framework for comparative analysis. A detailed configuration of the experimental parameters is provided in Table 2.

Table 2. Parameters for the experiment.

Parameter	Description	Value
N	Total number of nodes	64
byzNo	Number of Byzantine nodes	0 to 20
strategy	Byzantine strategy	forking/silence
timeout	Maximum waiting time for the next view	1000 ms
hasher	The hash algorithm used	sha3_256
max round	The maximum number of running epochs	5000
bsize	Number of transactions included in a block	400
r_0	Basic reward value	8
p_0	Basic penalty value	2
α	Punishment growth coefficient	0.3
β	Reward decay rate	0.2
k_1	Historical reputation weight	0.6
k_2	Real-time behavior weight	0.4

5.8. Experimental Metrics

Latency refers to the time interval between a client initiating a request and receiving a confirmation response.

TPS (Transactions Per Second): An indicator of a system's capacity to process transactions within a specified time frame, defined as the number of successfully processed transactions per second.

Block Interval [19] (BI): It measures the average number of view switches required for a block in a blockchain to be generated and submitted. It is mathematically defined as follows:

$$BI = \lim_{v \rightarrow \infty} \frac{\sum_{i=1}^{C(v)} L_i}{C(v)} \quad (22)$$

where L_i represents the number of views experienced by the i -th block, measured from the start to the submission.

Chain Growth Rate [19] (CGR): It refers to the growth rate of submitted blocks during the long-term operation of the system. The equation for its calculation is as follows:

$$CGR = \lim_{v \rightarrow \infty} \frac{C(v)}{v} \quad (23)$$

where $C(v)$ represents the total number of successfully submitted blocks during the first v views.

6. Results and Discussion

6.1. Under Silence Attack

As illustrated in Figure 8, the RVR algorithm exhibits superior delay management in the context of silence attacks when juxtaposed with the HotStuff and FastHotStuff algorithms. In comparison to the HotStuff benchmark, RVR achieves a 37.88% reduction in request latency. Regarding throughput, as depicted in Figure 9, the decline in transactions per second (TPS) for RVR is more gradual, maintaining 53.09% of the baseline TPS (achieved with no Byzantine nodes) when HotStuff's TPS reaches its minimum value. This improvement can be attributed to RVR's dynamic reputation mechanism, which effectively reduces the likelihood of the re-election of malicious leaders—a notable vulnerability identified in other consensus algorithms within blockchain technology [16,18].

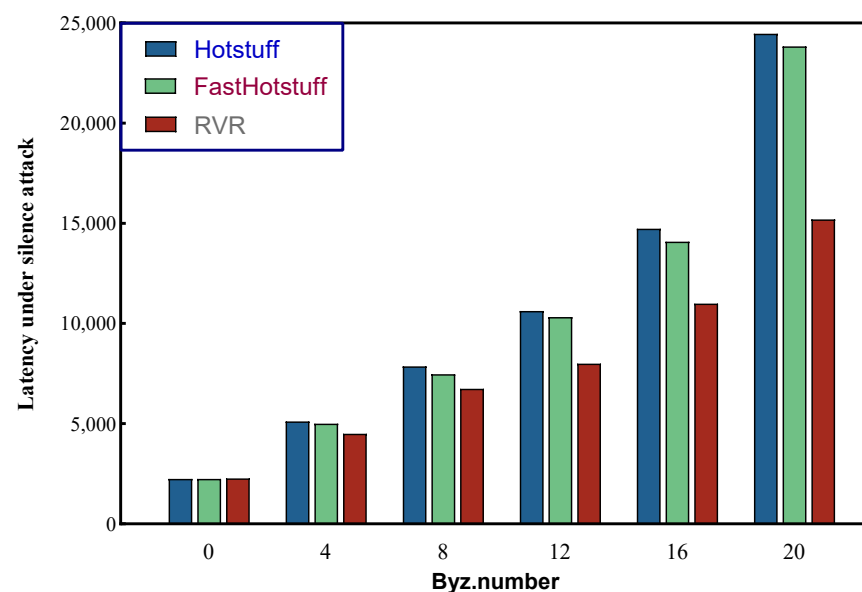


Figure 8. Delay performance under silence attack.

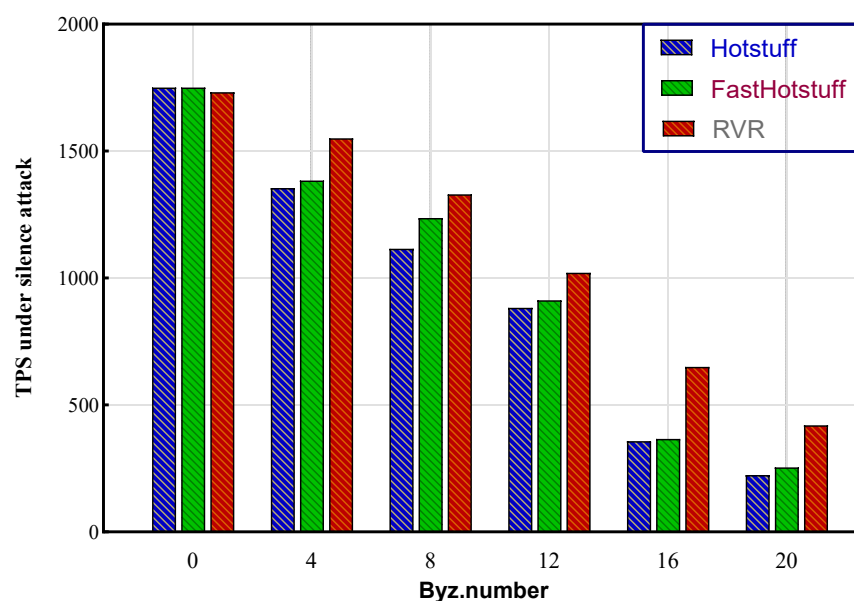


Figure 9. TPS performance under silence attack.

HotStuff and FastHotStuff utilize a fixed leader rotation mechanism, which enables Byzantine nodes to consistently impede block submissions during silence attacks. In

contrast, RVR employs a verifiable random function (VRF)-based selection process that effectively excludes leaders with lower reputation weights from the electoral process, thereby alleviating performance degradation.

Figures 10 and 11 demonstrate that RVR incorporates an optimized mechanism for filtering Byzantine nodes, resulting in diminished fluctuations in key performance indicators such as block inclusion (BI) and consensus gain rate (CGR). As illustrated in Figure 11, the minimum BI value for RVR is only 59.25% of that observed in HotStuff, thereby substantiating the effectiveness of its Byzantine fault-tolerant mechanism.

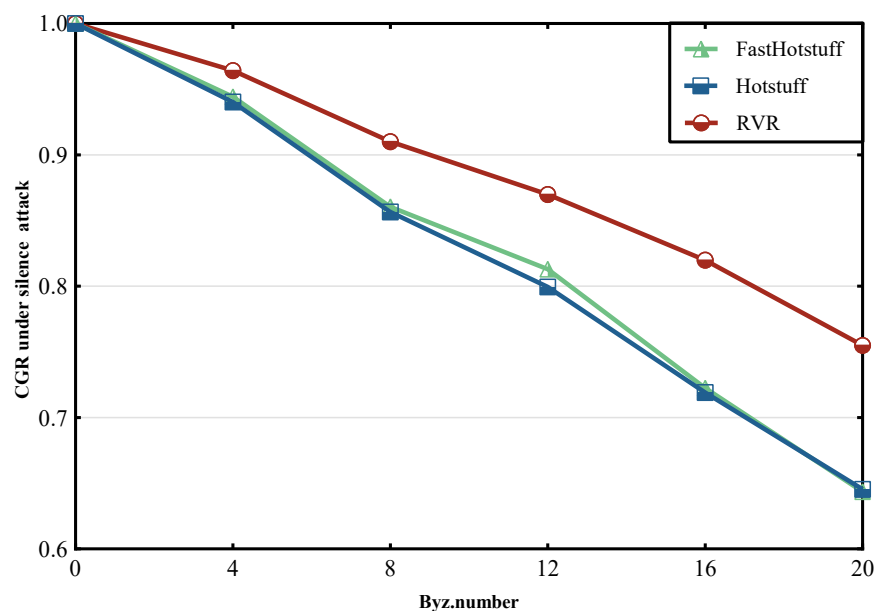


Figure 10. CGR performance under silence attack.

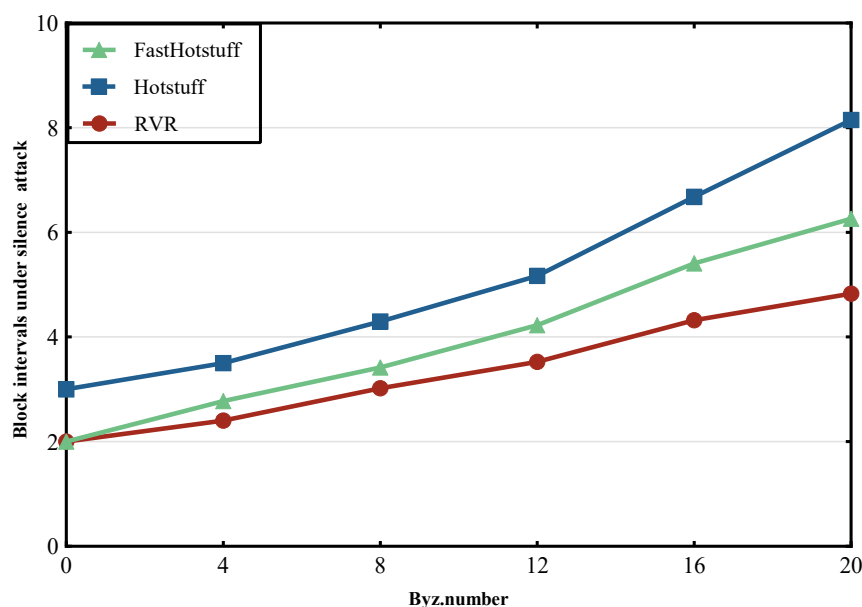


Figure 11. BI performance under silence attack.

6.2. Under Forking Attack

As depicted in Figure 12, under a forking attacks, RVR achieves latency values as low as 47.59% of HotStuff's value, while consistently sustaining stable transactions per second (TPS), as shown in Figure 13. This methodology surpasses the performance of FastHot-

Stuff's AggQC mechanism, which, although it successfully mitigates communication overhead, does not effectively prevent malicious leaders from creating conflict chains.

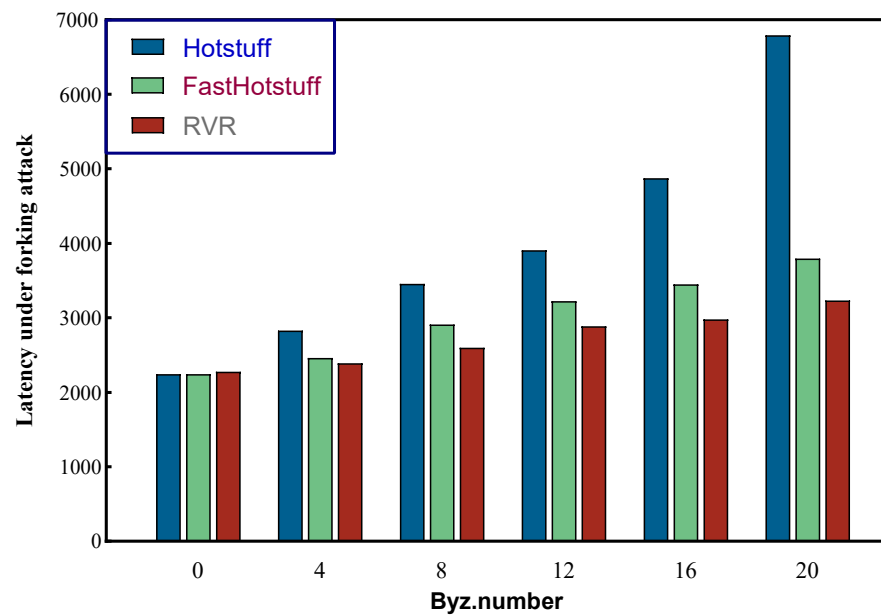


Figure 12. Delay performance under forking attack.

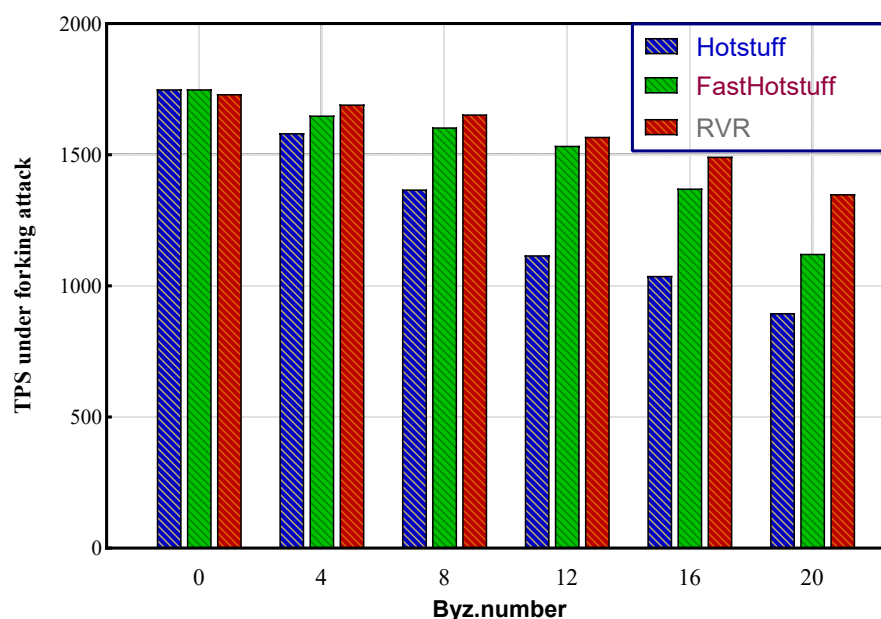


Figure 13. TPS performance under forking attack.

HotStuff requires the production of three sequential blocks to attain finality (Figure 3). Nonetheless, Byzantine leaders may take advantage of this stipulation to generate forks. The RVR protocol mitigates this concern by implementing a behavioral weight penalty, as described in Equation (16), which effectively reduces the standing of nodes that instigate forks. This mechanism significantly decreases their likelihood of being elected in an exponential fashion.

FastHotStuff partially mitigates challenges related to latency and throughput through the aggregation of quorum certificates (aggQCs) [18]. However, its absence of reputation awareness leads to instability in transactions per second (TPS) at elevated Byzantine fault ratios, as illustrated in Figure 13. In contrast, the reputation-driven architecture of RVR facilitates a more stable system throughput, as depicted in Figure 13, achieving a

maximum enhancement of 50.66% relative to HotStuff. Further examination, as shown in Figures 14 and 15, reveals that RVR (with a block confirmation efficiency index of $BI = 3.81$) exhibits a maximum increase of 24.10% compared to HotStuff ($BI = 5.02$). Additionally, the chain growth rate ($CGR = 0.71$) demonstrates a maximum improvement of 61.36%.

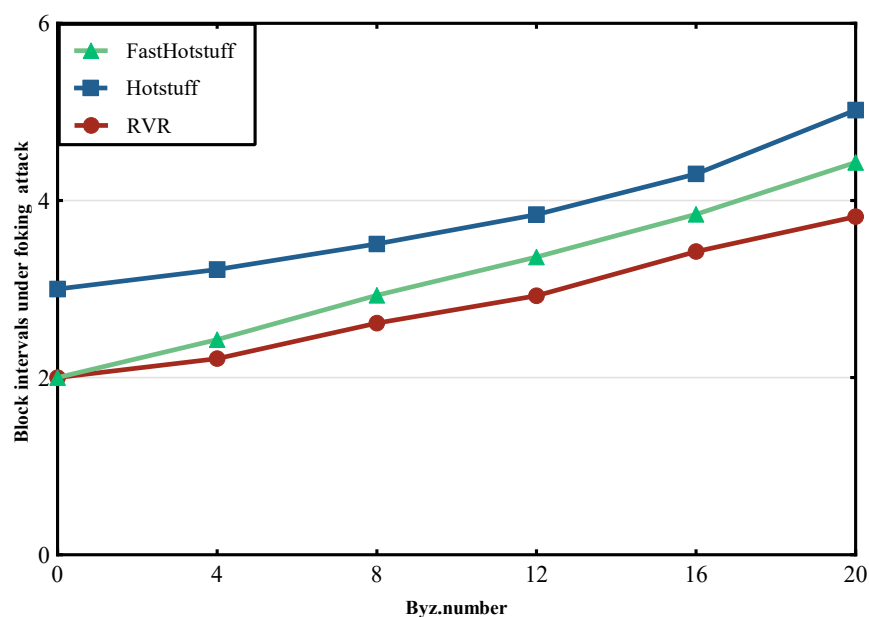


Figure 14. BI performance under forking attack.

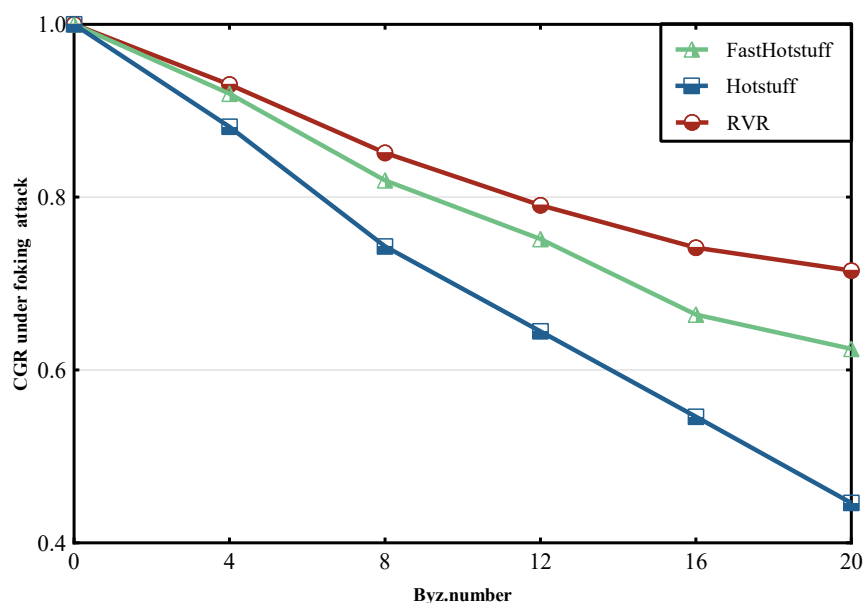


Figure 15. CGR performance under forking attack.

6.3. Discussion

The experimental validation confirmed RVR's paradigm-shifting capabilities in Byzantine-resilient consensus. Under a silence attacks, RVR achieved a remarkable 37.88% latency reduction compared to state-of-the-art HotStuff [16], demonstrating its unique capacity to mitigate targeted leader disruption. This breakthrough originated from RVR's innovative integration of dynamic reputation evaluation with verifiable random functions—a dual-mechanism approach that fundamentally addresses critical limitations in existing frameworks. Unlike Kumar et al.'s domain-specific model [21], which lacks adaptive reputation calibration, RVR's exponential decay function (Equation (18)) enables real-time

identification and suppression of malicious nodes through continuous behavior monitoring. Furthermore, while HotStuff family protocols [16,18] remain vulnerable to prediction-based targeting due to their deterministic leader sequences, RVR's cryptographically secure VRF-driven election ensures truly unpredictable coordinator selection, eliminating this attack vector entirely.

For forking attacks, RVR delivers equally impressive 50.66% throughput enhancement over HotStuff [16], establishing new standards for fork-resistant consensus in decentralized energy trading. RVR proactively prevents forks through behavior weight penalties (Equation (16)) that actively demote nodes initiating forking attacks, while FastHotStuff's AggQC mechanism [18] focuses exclusively on latency optimization and fails to address core fork vulnerabilities. Through its elegant integration of reputation-weighted quorum certificates with VRF-based view transitions, RVR maintains deterministic consistency while avoiding the prohibitive $O(n^2)$ messaging overhead that plagues classical BFT systems like PBFT [9]. This architectural innovation successfully resolves the fundamental tension between fork resistance and scalability identified in Gai et al.'s landmark analysis of chained consensus vulnerabilities [19], while simultaneously outperforming application-specific solutions such as Puthal et al.'s token-based approaches [20] through its universal design framework.

The combined security and efficiency optimization achieved by RVR represents a significant advancement for smart grid applications, where its linear communication complexity enables practical deployment in large-scale networks while providing robust protection against sophisticated Byzantine strategies that cripple conventional consensus mechanisms.

7. Conclusions

7.1. Technological Advancements and Implications

The RVR framework successfully reconciles Byzantine fault tolerance with operational efficiency within blockchain-based smart grid systems. By mitigating vulnerabilities associated with leader election in protocols such as HotStuff [16] and FastHotStuff [18], RVR establishes a scalable and secure foundation for advanced energy trading systems. Empirical findings indicate that RVR fulfills two primary objectives:

(1) Improved Byzantine Fault Tolerance: Through the incorporation of dynamic reputation assessment alongside verifiable randomness, RVR achieves a throughput enhancement of 50.66% relative to HotStuff during forking attacks, while also optimizing latency by 37.88% in the context of silence attacks.

(2) Broad Applicability: In contrast to prior research that has predominantly concentrated on single-device scenarios [9,16], RVR's multi-dimensional reputation model is designed to accommodate diverse energy trading environments.

These advancements carry substantial implications for the practical implementation of smart grids.

(1) Security: RVR's hybrid mechanism provides a robust safeguard for ensuring transaction authenticity within open peer-to-peer networks.

(2) Scalability: The algorithm's linear communication complexity of $O(N)$ and its dynamic weight-driven election mechanism render it well-suited for blockchain consensus networks characterized by fluctuating participant behavior.

7.2. Limitations and Directions for Future Research

The proposed framework is subject to several limitations that will guide future research endeavors.

(1) Assumption of Homogeneous Energy Types: The current framework presupposes uniform energy types (e.g., electricity). Future research will aim to extend the RVR model

to incorporate multi-energy coupled trading scenarios, including the integration of heat and hydrogen markets [31].

(2) Centralized Reputation Initialization: The reputation ledger's reliance on third-party key issuance partially undermines the principles of decentralization. Future work will focus on developing a fully decentralized mechanism for reputation initialization.

(3) Practical Constraints of Experimental Scale: The existing experiments are limited by the resource configurations of the simulation environment (e.g., a deployment of 64 nodes), which constrains the validation of RVR's performance in ultra-large-scale networks (exceeding 500 nodes). Although the linear communication complexity, $O(N)$, theoretically supports scalability, the impacts of dynamic load balancing and inter-regional communication delays necessitate further exploration. Future initiatives will include optimizing algorithmic parallelism (e.g., through sharding technology [32]), scaling the node count to over 1000 on distributed cloud platforms, and assessing its applicability within city-level smart grids.

(4) Centralized Key Distribution: The current framework relies on a third-party authority for consensus private key initialization, which contradicts the decentralized nature of blockchain systems. Future research will integrate decentralized key management protocols, such as distributed key generation (DKG) [33], to eliminate single points of trust.

Future research will concentrate on (1) integrating machine learning for real-time reputation calibration, (2) extending the framework to encompass multi-energy ecosystems, (3) investigating decentralized reputation initialization schemes to enhance system autonomy, and (4) implementing decentralized key distribution mechanisms to align with blockchain's trustless principles.

Author Contributions: Conceptualization, H.W. and X.L.; methodology, H.W.; software, H.W.; validation, H.W., X.L. and J.C.; formal analysis, X.L.; investigation, H.W.; resources, J.C.; data curation, H.W.; writing—original draft preparation, H.W.; writing—review and editing, X.L.; visualization, H.W.; supervision, X.L.; funding acquisition, X.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The raw/processed data required to reproduce these findings cannot be shared at this time as the data also forms part of an ongoing study.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Ha, T.; Lee, D.; Lee, C.; Cho, S. VCG auction mechanism based onblock chain in smart grid. In Proceedings of the 2021 International Conference on Information Networking (ICOIN), Jeju Island, Republic of Korea, 13–16 January 2021; pp. 465–468.
- Singla, S.; Dua, A.; Kumar, N.; Tanwar, S. Blockchain-based efficientenergy trading scheme for smart-grid systems. In Proceedings of the 2020 IEEE Globecom Workshops (GC Wkshps), Taipei, Taiwan, 7–11 December 2020; pp. 1–6.
- Doan, H.T.; Cho, J.; Kim, D. Peer-to-peer energy trading in smart grid through blockchain: A double auction-based game theoretic approach. *IEEE Access* **2021**, *9*, 49206–49218. [\[CrossRef\]](#)
- Maharjan, S.; Zhang, Y.; Gjessing, S.; Tsang, D.H.K. User-Centric Demand Response Management in the Smart Grid With Multiple Providers. *IEEE Trans. Emerg. Top. Comput.* **2017**, *5*, 494–505. [\[CrossRef\]](#)
- Zhang, Y.; Wang, A.; Zhang, H. Overview of smart grid development in China. *Power Syst. Prot. Control* **2021**, *49*, 180–187. [\[CrossRef\]](#)
- Xie, J.; Tang, H.; Huang, T.; Yu, F.R.; Xie, R.; Liu, J.; Liu, Y. A survey of blockchain technology applied to smart cities: Research issues and challenges. *IEEE Commun. Surv. Tutor.* **2019**, *21*, 2794–2830. [\[CrossRef\]](#)
- Wang, T.; Guo, J.; Ai, S.; Cao, J. RBT: A distributed reputation system for blockchain-based peer-to-peer energy trading with fairness consideration. *Appl. Energy* **2021**, *295*, 117056. [\[CrossRef\]](#)
- Micali, S.; Rabin, M.; Vadhan, S. Verifiable random functions. In Proceedings of the 40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039), New York, NY, USA, 17–19 October 1999; pp. 120–130. [\[CrossRef\]](#)

9. Castro, M.; Liskov, B. Practical Byzantine fault tolerance and proactive recovery. *ACM Trans. Comput. Syst.* **2002**, *20*, 398–461. [[CrossRef](#)]
10. Jakobsson, M.; Juels, A. Proofs of Work and Bread Pudding Protocols (Extended Abstract). In *Secure Information Networks*; Springer: Berlin/Heidelberg, Germany; Boston, MA, USA, 1999; pp. 258–272.
11. Kotla, R.; Alvisi, L.; Dahlin, M.; Clement, A.; Wong, E. Zyzzyva: Speculative Byzantine fault tolerance. *ACM Trans. Comput. Syst.* **2010**, *27*, 7. [[CrossRef](#)]
12. Miller, A.; Xia, Y.; Croman, K.; Shi, E.; Song, D. The Honey Badger of BFT Protocols. In Proceedings of the ACM SIGSAC Conference on Computer & Communications Security, Vienna, Austria, 24–28 October 2016; ACM: New York, NY, USA, 2016; pp. 31–42.
13. Zhang, M.; Li, J.; Chen, Z.; Chen, H.; Deng, X. An Efficient and Robust Committee Structure for Sharding Blockchain. *IEEE Trans. Cloud Comput.* **2023**, *11*, 2562–2574. [[CrossRef](#)]
14. Pass, R.; Shi, E. *Thunderella: Blockchains with Optimistic Instant Confirmation*; Springer: Cham, Switzerland, 2018; Volume 10821, pp. 3–33.
15. Gueta, G.G.; Abraham, I.; Grossman, S.; Malkhi, D.; Pinkas, B.; Reiter, M.; Seredinschi, D.-A.; Tamir, O.; Tomescu, A. SBFT: A Scalable and Decentralized Trust Infrastructure. In Proceedings of the 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Portland, OR, USA, 24–27 June 2019; pp. 568–580. [[CrossRef](#)]
16. Yin, M.; Malkhi, D.; Reiter, M.K.; Gueta, G.G.; Abraham, I. HotStuff: Bconsensus with linearity and responsiveness. In Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, Toronto, ON, Canada, 29 July–2 August 2019; Association for Computing Machinery: New York, NY, USA, 2019; pp. 347–356.
17. Boldyreva, A. Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme. In *Public Key Cryptography—PKC 2003*; Desmedt, Y.G., Ed.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2003; Volume 2567. [[CrossRef](#)]
18. Jalalzai, M.M.; Niu, J.; Feng, C.; Gai, F. Fast-HotStuff: A Fast and Robust BFT Protocol for Blockchains. *IEEE Trans. Dependable Secur. Comput.* **2024**, *21*, 2478–2493. [[CrossRef](#)]
19. Gai, F.; Farahbakhsh, A.; Niu, J.; Feng, C.; Beschastnikh, I.; Duan, H. Dissecting the Performance of Chained-BFT. In Proceedings of the 2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS), Washington, DC, USA, 7–10 July 2021; pp. 595–606. [[CrossRef](#)]
20. Puthal, D.; Malik, N.; Mohanty, S.; Kougianos, E.; Das, G. Everything you wanted to know about the blockchain: Its promise, components, processes, and problems. *IEEE Consum. Electron. Mag.* **2018**, *7*, 6–14. [[CrossRef](#)]
21. Kumar, N.; Aujla, G.; Das, A.; Conti, M. ECCAuth: A secure authentication protocol for demand response management in a smart grid system. *IEEE Trans. Ind. Inf.* **2019**, *15*, 6572–6582. [[CrossRef](#)]
22. Kaur, D.; Aujla, G.S.; Kumar, N.; Zomaya, A.Y.; Perera, C.; Ranjan, R. Tensor-based big data management scheme for dimensionality reduction problem in smart grid systems: SDN perspective. *IEEE Trans. Knowl. Data Eng.* **2018**, *30*, 1985–1998. [[CrossRef](#)]
23. Xiao, Y.; Zhang, N.; Lou, W.; Hou, Y.T. A Survey of Distributed Consensus Protocols for Blockchain Networks. *IEEE Commun. Surv. Tutor.* **2020**, *22*, 1432–1465. [[CrossRef](#)]
24. Cerf, R. The quasi species regime for the simple genetic algorithm with roulette wheel selection. *Adv. Appl. Probab.* **2017**, *49*, 903–926. [[CrossRef](#)]
25. Bianchi, C.; Langner, M.R.; Mishra, V.; Torcellini, P. *Accelerating the Adoption of Energy Efficiency and Renewables in Warehouses and Distribution Centers*; National Renewable Energy Laboratory (NREL): Golden, CO, USA, 2023. [[CrossRef](#)]
26. Kirst, C.; Timme, M.; Battaglia, D. Dynamic information routing in complex networks. *Nat. Commun.* **2016**, *7*, 11061. [[CrossRef](#)] [[PubMed](#)]
27. Sansone, C.; Vento, M. Signature Verification: Increasing Performance by a Multi-Stage System. *Pattern Anal. Appl.* **2000**, *3*, 169–181. [[CrossRef](#)]
28. Hsueh, S.-C.; Kuo, C.-H. Effective Matching for P2P Lending by Mining Strong Association Rules. In Proceedings of the 3rd International Conference on Industrial and Business Engineering (ICIBE '17), Sapporo, Japan, 17–19 August 2017; Association for Computing Machinery: New York, NY, USA, 2017; pp. 30–33. [[CrossRef](#)]
29. Li, J.; Wu, J.; Chen, L. Block-secure: Blockchain based scheme for secure P2P cloud storage. *Inf. Sci.* **2018**, *465*, 219–231. [[CrossRef](#)]
30. Blue, J.; Condell, J.; Lunney, T. A Review of Identity, Identification & Authentication. *Int. J. Inf. Secur. Res.* **2018**, *8*, 794–804. [[CrossRef](#)]
31. Zhang, R.; Bu, S.; Li, G. Multi-market P2P trading of cooling–heating–power–hydrogen integrated energy systems: An equilibrium-heuristic online prediction optimization approach. *Appl. Energy* **2024**, *367*, 123352. [[CrossRef](#)]

32. Yu, G.; Wang, X.; Yu, K.; Ni, W.; Zhang, J.A.; Liu, R.P. Survey: Sharding in Blockchains. *IEEE Access* **2020**, *8*, 14155–14181. [\[CrossRef\]](#)
33. Kate, A.; Goldberg, I. Distributed Key Generation for the Internet. In Proceedings of the 2009 29th IEEE International Conference on Distributed Computing Systems, Montreal, QC, Canada, 22–26 June 2009; pp. 119–128. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.